

Neo4j & Apache Cassandra

An Academic Deep Dive:
Graph Databases vs. Wide-Column Stores

Lâm Khánh Trình

Ho Chi Minh City University of Technology (HCMUT)
Faculty of Computer Science & Engineering

May 5, 2026

Table of Contents

- 1 The NoSQL Landscape
- 2 Neo4j: The Graph Database
- 3 Apache Cassandra: The Wide-Column Store
- 4 Deep Comparison
- 5 Polyglot Persistence
- 6 Emerging Trends & Research Directions
- 7 Summary & Conclusions

The Rise of NoSQL: Historical Context

The Relational Era (1970s – 2000s)

- Edgar Codd's relational model dominated for decades
- SQL databases: strong consistency, rigid schemas, vertical scaling
- The Web 2.0 explosion shattered their assumptions:
 - Billions of users generating semi-structured data at high velocity
 - Social graphs, time-series logs, documents — all ill-fitting
 - Cost of vertical scaling became prohibitive

Four NoSQL Families

Key-Value	Redis, DynamoDB
Document	MongoDB, CouchDB
Wide-Column	Cassandra , HBase
Graph	Neo4j , JanusGraph

Each family excels at a specific *access pattern*. Choosing wrong leads to fundamental architectural pain.

The NoSQL Response (2004 – present)

- Google Bigtable (2004), Amazon Dynamo (2007) papers ignited the movement
- “Not Only SQL” — horizontal scale, flexible

CAP Theorem: The Fundamental Trade-off

CAP Theorem (Brewer, 2000)

A distributed system can guarantee **at most two** of the three:

- C Consistency** — every read receives the most recent write or an error
- A Availability** — every request receives a (possibly stale) response
- P Partition Tolerance** — system continues despite network partitions

In practice, P is mandatory for any distributed system. The real choice is C vs. A during a partition.

Neo4j Positioning: CA / CP

Prioritizes **Consistency**. Causal Clustering uses the Raft

Cassandra Positioning: AP

Prioritizes **Availability** and **Partition Tolerance**. Accepts *Eventual Consistency* by default. During a network partition, Cassandra prefers to serve potentially stale data rather than refuse the request. The key

innovation: **Tunable Consistency** allows operators to dial the CP/AP balance per query using consistency levels (ONE, QUORUM, ALL).

Deep Dive: PACELC Extension

The PACELC model (Abadi, 2012) refines CAP: *even without* a partition, systems trade off **Latency** vs. **Consistency**. Cassandra is PA/EL (prioritize availability + low latency); Neo4j is PC/EC (prioritize consistency).

ACID vs. BASE: Competing Consistency Philosophies

ACID (Neo4j)

- **Atomicity** — transaction is all-or-nothing
- **Consistency** — data always moves from valid state to valid state
- **Isolation** — concurrent transactions do not interfere
- **Durability** — committed data survives failures

Neo4j enforces full ACID on every graph transaction, including multi-hop relationship writes.

BASE (Cassandra)

- **Basically Available** — system is always responsive
- **Soft state** — state may change over time without input
- **Eventually consistent** — all replicas converge to same value *eventually*

Cassandra introduced *Lightweight Transactions* (Paxos-based) for conditional writes when strict ordering is required, but at significant performance cost.

Data Models: A Taxonomy

Dimension	Graph (Neo4j)	Wide-Column (Cassandra)
Core abstraction	Nodes & typed Relationships	Rows with dynamic column families
Schema	Flexible (schema-optional)	Per-table schema, strictly enforced
Primary query	Pattern matching, traversal	Partition key lookup
Joins	Native, $O(1)$ per hop	Forbidden (application-side only)
Relationships	First-class citizens	Encoded as denormalized columns
Ordering	Graph topology	Clustering column defines disk order

Neo4j: History & Origins

- Founded in **2007** by Emil Eifrem, Johan Svensson, and Peter Neubauer in Malmö, Sweden — initially as a backend for a content management system
- Open-sourced under GPLv3 in 2010; community edition remains free
- Introduced the **Cypher query language** (2011), later donated to the openCypher project (2015) for cross-vendor standardization
- Version **4.x** introduced multi-database and Fabric sharding; **5.x** unified clustering under the Raft-based Causal Clustering model
- Currently the **#1 ranked graph database** on DB-Engines and the de-facto standard for property graph systems

Key Milestones

2007	Company founded
2010	First stable release
2011	Cypher language introduced
2013	Enterprise clustering
2015	openCypher project
2017	Causal Clustering (Raft)
2020	Neo4j 4.0 (multi-DB)
2022	AuraDB (managed cloud)
2023	Neo4j 5.x (unified arch.)

The Property Graph Model

Four primitive elements:

1. Nodes (Vertices)

Represent **entities** (nouns). Can carry one or more *Labels* (e.g., :Student, :Course) and arbitrary key-value *Properties* (e.g., name: "Trinh", gpa: 3.8).

2. Relationships (Edges)

Represent **connections** (verbs). Always *directed* (but can be traversed both ways). Have exactly one *Type* (e.g., :ENROLLED_IN) and can also carry *Properties* (e.g., since: 2023).

3. Labels & 4. Properties

Labels: categorization mechanism; one node can have many labels. **Properties:** schema-optional key-value pairs stored directly on nodes/edges — no separate tables

Why Graph Matters for Relationships

In a relational database, representing:

Alice → FRIEND_WITH → Bob

requires a friendships JOIN table and an expensive index scan. Adding a *property* to that friendship (e.g., since: 2020) requires altering the JOIN table. In

Neo4j, relationships are first-class objects stored with their own properties. The relationship record directly embeds pointers to both endpoint nodes — no index needed.

Key Differentiator

Relationships in Neo4j are **not derived at query time**. They are *pre-materialized on disk*.

Index-Free Adjacency: Neo4j's Performance Secret

How traditional databases join:

- SQL uses **B-Tree indexes** over foreign key columns
- To resolve a JOIN: hash/merge probe on the index — $O(\log N)$ per lookup
- Each additional hop in a multi-join query multiplies cost
- For a 6-hop social graph traversal on 1 billion nodes: **infeasible**

How Neo4j traverses:

- Each node record contains a **physical pointer** to its first relationship record
- Each relationship record contains pointers to: the start node, end node, and the *next relationship* of each node (a doubly-linked list)

Deep Dive: Storage Record Layout

Node record (15 bytes):

[inUse|nextRelId|nextPropId|labels]

Relationship record (34 bytes):

[inUse|firstNode|secondNode|
relType|firstPrevRelId|
firstNextRelId|secondPrevRelId|
secondNextRelId|nextPropId]

Fixed-size records enable **direct byte-offset addressing**: finding record #N is simply a seek to byte $N \times 15$ — no heap scan, no index lookup.

Complexity comparison for a k -hop query:

SQL: $O(k \cdot \log N)$

Neo4j: $O(k \cdot d)$

Neo4j Architecture: Deployment Modes

Single Instance

Full ACID. Reads and writes on same machine. Suitable for development, analytics workloads, or small production deployments with low HA requirements.

Causal Clustering (HA)

Core Servers (typically 3 or 5): participate in Raft consensus. All writes go through Raft leader; a majority must acknowledge before commit.

Read Replicas: asynchronously replicate from Core Servers; serve read queries; can lag behind by a small delta.

Causal Consistency: a client receives a *bookmark* after a write; subsequent reads present that bookmark to guarantee they see at least that write.

Deep Dive: Raft Consensus in Neo4j

Raft ensures that at most one leader exists per term:

- 1 Leader receives a write transaction
- 2 Appends to its local log and broadcasts to followers
- 3 Transaction is committed once a **quorum** ($\lfloor n/2 \rfloor + 1$) acknowledges
- 4 Leader responds to client; followers apply asynchronously

Failure scenario: if the leader fails, followers elect a new one via randomized timeouts. No data is lost if a majority survived.

With 3 Core Servers: tolerates **1 failure**.

With 5 Core Servers: tolerates **2 failures**.

Neo4j Fabric (Sharding)

Neo4j Storage Engine & Transaction Lifecycle

Storage layers:

- 1 **Transaction Log** — every write is first appended to the WAL (Write-Ahead Log) for durability, analogous to PostgreSQL's WAL
- 2 **Node/Relationship/Property Stores** — fixed-size record files on disk (mmap'd into memory for fast access)
- 3 **Label Index Store** — allows efficient lookup of all nodes with a given label
- 4 **Schema Indexes** — B-Tree, Full-Text (Lucene), or Point indexes over node/relationship properties
- 5 **Page Cache** — configurable in-memory cache of store pages; the primary performance lever

Deep Dive: MVCC & Locking

Neo4j uses a **pessimistic locking** model (not MVCC like PostgreSQL):

- Shared locks on *read* operations
- Exclusive locks on *write* operations (node/relationship/property records)
- Deadlock detection via cycle-detection in the lock manager wait graph
- Phantom reads are prevented by *label scan locks*

Implication: high-concurrency write workloads on the *same* nodes/relationships can produce lock contention. Design the data model to minimize hot nodes.

Cypher Query Language: Design Philosophy

ASCII-Art Syntax

Cypher's defining insight: **graph patterns look like drawings.**

- `(n)` — a node variable `n`
- `(n:Label)` — node with a label
- `(n {prop: val})` — node with a property filter
- `-[:TYPE]->` — directed relationship
- `-[:TYPE*1..3]->` — variable-length path (1 to 3 hops)
- `(a)--(b)` — undirected relationship

These compose into **path patterns** that read like sentences:
`(alice)-[:FRIEND]->(bob)-[:WORKS_AT]->(acme)`

Core Cypher Clauses

<code>MATCH</code>	Pattern search
<code>OPTIONAL MATCH</code>	Left outer join
<code>WHERE</code>	Predicate filtering
<code>RETURN</code>	Project output
<code>CREATE</code>	Insert nodes/rels
<code>MERGE</code>	Upsert
<code>SET / REMOVE</code>	Update properties
<code>DELETE / DETACH</code>	Remove data
<code>WITH</code>	Pipeline stages
<code>UNWIND</code>	Flatten lists
<code>CALL</code>	Invoke procedures
<code>FOREACH</code>	Iterate side-effects

openCypher Standard

Since 2015, Cypher's specification is maintained by the openCypher community, COI (ISO/IEC JTC1/SC30).

Cypher: CRUD Operations

Create nodes and relationships:

```
// Create two students and a course
CREATE (t:Student {id: "SV001", name: "Trinh",
                    gpa: 3.8, faculty: "CS"})
CREATE (k:Student {id: "SV002", name: "Khoa",
                    gpa: 3.5, faculty: "CS"})
CREATE (c:Course {code: "C03021",
                  name: "Database Systems"})

// Create typed, property-bearing relationship
CREATE (t)-[:STUDIES_WITH {since: 2022,
                             project: "NoSQL Research"}]->(k)
CREATE (t)-[:ENROLLED_IN {semester: "HK2"}]->(c)
CREATE (k)-[:ENROLLED_IN {semester: "HK2"}]->(c)
```

Update and delete:

```
// Update a property
MATCH (s:Student {id: "SV001"})
SET s.gpa = 3.9, s.honors = true

// Remove a property
MATCH (s:Student {id: "SV001"})
REMOVE s.honors

// Delete a node and all its relationships
MATCH (s:Student {id: "SV002"})
DETACH DELETE s
```

MERGE Semantics

MERGE is Neo4j's upsert: it matches the full pattern, and if no match exists, creates it atomically. Essential for idempotent data loading pipelines.

Cypher: Advanced Pattern Matching

Variable-length path traversal:

```
// Friends within 3 hops of Trinh
// who share the same faculty
MATCH (me:Student {name: "Trinh"})
MATCH (me)-[:FRIEND*1..3]-(network:
    Student)
WHERE network.faculty = me.faculty
    AND network <> me
RETURN network.name,
    length(shortestPath(
        (me)-[:FRIEND*]-(network)
    )) AS hops
ORDER BY hops ASC
LIMIT 10
```

Aggregation and collection:

```
// Top courses by enrollment within
    network
MATCH (me:Student {name: "Trinh"})
MATCH (me)-[:FRIEND*1..2]-(peer:Student)
MATCH (peer)-[:ENROLLED_IN]->(c:Course)
```

Shortest path algorithms:

```
// Shortest friendship path between
// two students
MATCH (a:Student {id: "SV001"}),
    (b:Student {id: "SV099"})
MATCH p = shortestPath(
    (a)-[:FRIEND*]-(b)
)
RETURN [n IN nodes(p) | n.name]
    AS path,
    length(p) AS degrees
```

Deep Dive: Query Planning

Cypher queries are compiled by the **Cypher Planner** into a physical execution plan (observable via `EXPLAIN / PROFILE`). The planner uses statistics (node/relationship counts) and index availability to choose between:

NodeIndexScan, NodeByLabelScan, Expand, ShortestPath, NodeHashJoin, etc.

Cypher: Graph Algorithms & APOC

Neo4j Graph Data Science (GDS) library:

```
// Project an in-memory graph
CALL gds.graph.project(
  'social-graph',
  'Student',
  'FRIEND'
)

// Run PageRank to find influence
CALL gds.pageRank.stream('social-graph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS
      name,
      score
ORDER BY score DESC LIMIT 10

// Community detection (Louvain)
CALL gds.louvain.stream('social-graph')
YIELD nodeId, communityId
RETURN communityId,
      count(*) AS members
ORDER BY members DESC
```

GDS Algorithm Categories

Centrality	PageRank, Betweenness
Community	Louvain, LPA, WCC
Path	Dijkstra, A*, Yen's k-shortest
Similarity	Node2Vec, cosine, Jaccard
Link Prediction	Adamic-Adar, Common Neighbors
Embeddings	FastRP, GraphSAGE

APOC (Awesome Procedures on Cypher)

APOC provides 450+ utility procedures for:

- Data import (CSV, JSON, GraphQL)
- Dynamic Cypher execution
- Date/time manipulation
- Graph refactoring
- Batch processing with

Neo4j Indexing Strategies

Schema Indexes (on node properties):

- **B-Tree Index** — range queries, equality, ORDER BY. Default for most property types.
- **Full-Text Index** — powered by Apache Lucene; supports CONTAINS, fuzzy matching, relevance scoring. Ideal for search on string properties.
- **Point Index** — spatial data types; supports distance and bounding-box queries.
- **Composite Index** — across multiple properties; used when WHERE filters multiple fields simultaneously.
- **Token Lookup Index** — auto-created; enables fast NodeByLabelScan.

Relationship indexes: from Neo4j 4.3+, indexes can be created on relationship properties as well.

Deep Dive: Index vs. Index-Free Adjacency

It is important to understand when each mechanism fires:

- **Schema indexes** are used as *entry points* — to find the first node(s) in a query (e.g., MATCH (s:Student {id:"SV001"}))
- Once the entry node is found, all subsequent -[:REL]-> traversals use **index-free adjacency** (pointer following)

This hybrid design combines the best of both worlds: fast property lookup via B-Tree + fast graph traversal via physical pointers.

Index Hint

Neo4j Use Cases & Anti-Patterns

[OK] Ideal Use Cases

- **Fraud ring detection** — identifying circular transaction patterns that look legitimate individually but form suspicious rings when traversed as a graph
- **Real-time recommendations** — “People who viewed X also viewed Y” traversed in real time across social and purchase graphs
- **Knowledge graphs** — connecting entities across multiple domains (research, enterprise ontologies, semantic web)

[!] Anti-Patterns

- **Bulk data logging** — massive flat record ingestion (metrics, events, access logs) where no relationship traversal occurs; Cassandra or ClickHouse is far better
- **Simple key-value lookups** — adding graph overhead to what is fundamentally a dictionary lookup
- **High-cardinality, schema-rigid data** — if data is perfectly tabular and queries never need joins, a relational database or columnar store outperforms
- **Supernode problem** — nodes with

Cassandra: History & Origins

- Originally developed by **Facebook** (2007–2008) to power the Inbox Search feature — required handling billions of messages with no downtime
- Open-sourced and donated to the **Apache Software Foundation** in 2009; graduated to Top-Level Project in 2010
- Architectural DNA: combines **Amazon Dynamo's** distributed hash ring (for availability) with **Google Bigtable's** data model (for structured storage)
- Major adopters include Netflix (replicas across 3 AWS regions), Apple (>75,000 nodes), Discord, Uber, and Instagram
- **DataStax Enterprise (DSE)** is the commercial distribution with enterprise add-ons; Astra DB is the managed cloud offering

Key Milestones

2007	Created at Facebook
2008	Deployed for Inbox Search
2009	Open-sourced (Apache)
2010	Apache TLP graduation
2012	CQL 3.0 released
2014	Lightweight Transactions
2016	SASI indexing
2020	Cassandra 4.0 (audit log)
2022	Cassandra 4.1 (pluggable)

Scale Fact

Apple runs Cassandra on over **75,000 nodes**, storing more than **10 petabytes** of data — the largest known Cassandra deployment.

The Wide-Column Data Model

Conceptual hierarchy:

- **Keyspace** — top-level container (like a relational database); defines replication strategy
- **Table** — collection of rows; each table has a declared schema
- **Row** — uniquely identified by its *primary key*; can have different non-null columns per row (wide columns)
- **Primary Key anatomy:**
 - **Partition Key** — determines which node(s) own this row via consistent hashing
 - **Clustering Columns** — determine the sorted order of rows *within* a partition on disk
 - **Regular Columns** — the data payload

Deep Dive: “Wide Column” Explained

The term refers to Bigtable’s original model where each row can have *millions of columns* — not the same as a relational table. In Cassandra, this manifests as clustering columns that create a sorted, sparse matrix structure within each partition.

A single Cassandra partition can hold up to **2 billion cells** (column values), making it ideal for time-series data where each event becomes a new row within the same partition.

Important Constraint

Cassandra does **not** support:

- JOINS between tables
- Subqueries

Cassandra: The Masterless Ring Architecture

Core architectural principles:

- **Peer-to-peer (P2P)** — no master node; every node is functionally identical
- **Consistent Hashing** — the partition key is hashed to a 64-bit token; the ring is divided into token ranges, each owned by a node
- **Replication Factor (RF)** — each partition is replicated on RF nodes for fault tolerance; typically RF=3
- **Coordinator** — any node can accept client requests and acts as coordinator for that request, routing sub-requests to the correct replicas
- **Virtual Nodes (vnodes)** — each physical node owns multiple token ranges (default 256), enabling automatic data balancing as

Deep Dive: Consistent Hashing

Problem solved: with naïve modular hashing ($\text{key} \% N$), adding/removing one node N invalidates $\approx N/(N+1)$ of all keys.

Solution: map both nodes and keys onto a fixed-size ring. When a node is added, it only takes responsibility for a *portion* of its neighbor's range. On average, only K/N keys are remapped (where K = total keys, N = nodes).

Virtual nodes extend this: each physical node's load can be tuned by assigning it more/fewer vnodes without moving data.

Zero Single Point of Failure

Because any node can coordinate any request, Cassandra has **no master to fail**. Rolling upgrades, node additions,

Gossip Protocol & Failure Detection

Gossip protocol mechanics:

- Each node periodically (every 1 second) selects 1–3 random peers and exchanges **state information** (token ownership, load, schema version, generation counter)
- Messages contain not only direct knowledge but also *known knowledge about other nodes* — rumors spread virally
- Convergence time: $O(\log N)$ rounds for full cluster awareness
- No central directory needed — each node builds a complete cluster view

Failure detection (Φ Accrual):

- Cassandra does *not* use binary alive/dead state
- Computes a continuous suspicion level Φ based

Deep Dive: Φ Accrual Detector

The Φ value is computed from inter-arrival times of heartbeat messages:

$$\Phi(t) = -\log_{10}(1 - F(t - t_{\text{last}}))$$

where F is the CDF of historical arrival intervals (modeled as a Gaussian distribution). As the gap between expected and actual heartbeat grows, Φ increases toward infinity.

This adaptive model avoids false positives in high-latency networks (e.g., cross-datacenter links) by adjusting the threshold automatically based on network conditions.

Key benefit: Gossip + Φ detection enables Cassandra to heal gracefully from transient network blips

Replication Strategies in Cassandra

SimpleStrategy

Places replicas on the next $RF - 1$ nodes clockwise on the ring, regardless of rack or datacenter. **Use only for single-datacenter deployments or development.**

NetworkTopologyStrategy (Production)

Specifies per-datacenter replication factors. Distributes replicas across different racks within each datacenter for maximum fault isolation.

Example: `datacenter1: 3, datacenter2: 3` means 6 total replicas across 2 DCs. A failure of one entire DC still allows the other DC to serve all data.

Deep Dive: Multi-DC Write Path

With `LOCAL_QUORUM` consistency:

- 1 Client writes to any coordinator in DC1
- 2 Coordinator routes write to all RF replicas in DC1 (synchronously)
- 3 Coordinator acknowledges to client once local quorum is met
- 4 Replication to DC2 happens **asynchronously** via hints

This achieves low-latency writes for the local DC while maintaining eventual consistency with the remote DC — ideal for active-active multi-region deployments.

Hinted Handoff

Cassandra Storage Engine: The Write Path

Step-by-step write path:

- ❶ **Commit Log** — write is appended to the per-node commit log (WAL) immediately. This is *sequential I/O* on disk — extremely fast. Ensures durability across crashes.
- ❷ **Memtable** — write is also written to an in-memory sorted structure (the Memtable). Reads on recently written data are served from here.
- ❸ **SSTable flush** — when the Memtable exceeds a threshold, it is *flushed* to disk as a new, immutable SSTable (Sorted String Table). Flush is also sequential.
- ❹ **Bloom Filter** — each SSTable has a Bloom filter in memory; used to quickly determine if a key *might* exist in that SSTable, avoiding unnecessary disk seeks.

Deep Dive: Tombstones

Cassandra does **not** delete data in-place. Deletions write a special record called a *tombstone* with a deletion timestamp. During compaction, rows whose tombstone timestamp exceeds `gc_grace_seconds` (default: 10 days) are actually removed.

Why 10 days? If a node was offline for 9 days and missed a delete, it must receive the tombstone via repair before the GC window expires. If the window expires first, the deleted data is resurrected (a “zombie” row).

Implication: designs that generate many tombstones (e.g., writing a “null” to update a cell) severely degrade read performance. Prefer TTL instead.

Cassandra Storage Engine: The Read Path

Step-by-step read path:

- ① Check **Row Cache** (if enabled and configured)
- ② Check **Memtable** — most recently written data served instantly
- ③ For each SSTable:
 - ① Check **Bloom filter** — skip SSTable if key definitely absent ($O(1)$)
 - ② Check **Partition Summary** — memory-resident sparse index of partition offsets
 - ③ Consult **Partition Index** — on-disk structure for exact byte offset of key
 - ④ Seek **SSTable data file** — read compressed, sorted data blocks
- ④ **Merge** results from Memtable and all SSTables; use timestamp to determine winner (last-write-wins)

Deep Dive: Read Repair

When consistency level requires reading from multiple replicas:

- ① Coordinator sends read to all required replicas
- ② Compares digest responses
- ③ If replicas disagree: fetches full data from all, picks latest timestamp, and *writes the correct value back* to the stale replicas
- ④ Returns the correct value to the client

This self-healing mechanism is called **Read Repair**. It runs in the foreground (blocking) at the configured probability, keeping replicas in sync without explicit repair jobs.

Compaction Strategy

Tunable Consistency: The AP/CP Dial

Consistency level governs how many replicas must respond:

Level	Semantics
ONE	Only 1 replica must respond (fastest, least consistent)
TWO / THREE	2 or 3 replicas must respond
QUORUM	Majority: $\lfloor RF/2 \rfloor + 1$ replicas
LOCAL_QUORUM	Quorum within local DC only
EACH_QUORUM	Quorum in <i>each</i> DC
ALL	All replicas must respond

Deep Dive: Strong Consistency Formula

With RF replicas, to achieve strong (linearizable) consistency, you need:

$$R + W > RF$$

where R = read CL replicas, W = write CL replicas.

Example (RF=3):

- Write: QUORUM (2) + Read: QUORUM (2) = 4 > 3 ✓ *Strong*
- Write: ONE (1) + Read: ONE (1) = 2 < 3 ✗ *Eventual*
- Write: ALL (3) + Read: ONE (1) = 4 > 3 ✓ *Strong (but low write availability)*

QUORUM + QUORUM is the typical production setting for most applications.

Query-First Data Modeling: The Core Principle

Cassandra's Cardinal Rule

Model your **tables around your queries**, not around your entities. Every query should be answerable by a single partition scan.

The modeling workflow:

- 1 Enumerate all application queries ("What does the app need to read?")
- 2 For each query: design a table whose partition key equals the query's primary filter
- 3 Choose clustering columns to sort data in the order the query needs
- 4 Accept data duplication as a *feature*, not a bug

Deep Dive: Why No JOINS?

A JOIN in a distributed system requires:

- 1 Broadcast query to all nodes holding relevant partitions
- 2 Shuffle large data volumes across the network
- 3 Merge results centrally

In a cluster with 100 nodes, this is a **fan-out of 100** network round-trips. Under high load, this becomes a distributed coordination nightmare.

Cassandra's answer: pre-compute the join at write time. Denormalize the data so each query only touches one partition. JOINS happen in the *application*, at write time, not at read time.

CQL: Keyspace & Table Design

Creating a Keyspace:

```
CREATE KEYSPACE university
WITH REPLICATION = {
    'class': 'NetworkTopologyStrategy',
    'datacenter1': 3,
    'datacenter2': 2
}
AND DURABLE_WRITES = true;

USE university;
```

Query-Driven Table: “Get all enrollments for a student”:

```
CREATE TABLE enrollments_by_student (
    student_id    uuid,
    enrolled_at   timestamp,
    course_code    text,
    course_name    text,
    semester       text,
    grade          float,
    PRIMARY KEY ((student_id),
```

Query-Driven Table: “Get top students in a course”:

```
-- Separate table for the reverse
query
CREATE TABLE students_by_course (
    course_code    text,
    gpa            float,
    student_id     uuid,
    student_name    text,
    semester       text,
    PRIMARY KEY ((course_code), gpa,
                student_id)
) WITH CLUSTERING ORDER BY (gpa DESC);
```

Two Tables, Two Queries

Both tables store the same relationship data (student ↔ course enrollment). This **denormalization** is intentional: each table is optimized for exactly one read query. Write operations update both tables atomically using a `BATCH` statement.

CQL: Advanced Features

TTL (Time-To-Live) for automatic expiry:

```
-- Session token expires in 3600 seconds
INSERT INTO session_tokens
    (user_id, token, created_at)
VALUES (uuid(), 'abc123xyz', toTimestamp
    (now()))
USING TTL 3600;

-- Per-column TTL
UPDATE user_preferences
USING TTL 86400
SET dark_mode = true
WHERE user_id = 550e8400-e29b-41d4-a716
    -446655440000;
```

Lightweight Transactions (Paxos):

```
-- Conditional insert (compare-and-set)
INSERT INTO accounts (id, email, balance
    )
VALUES (uuid(), 'trinh@hcmut.edu.vn',
    1000)
```

Collections (List, Set, Map):

```
CREATE TABLE student_profile (
    id        uuid PRIMARY KEY,
    name      text,
    tags      set<text>,
    scores    list<float>,
    metadata  map<text, text>
);

-- Append to list
UPDATE student_profile
SET scores = scores + [3.8]
WHERE id = ?;

-- Add to set
UPDATE student_profile
SET tags = tags + {'honors', 'research
    '}
WHERE id = ?;
```

Deep Dive: LWT Cost

CQL: Time-Series Data Modeling

Sensor data model — classic time-series:

```
CREATE TABLE iot_sensor_readings (  
    device_id    uuid,  
    -- Bucket by day to control  
    partition size  
    day          date,  
    recorded_at  timestamp,  
    temperature  float,  
    humidity     float,  
    pressure     float,  
    PRIMARY KEY ((device_id, day),  
                recorded_at)  
) WITH CLUSTERING ORDER BY (recorded_at  
    DESC)  
    AND default_time_to_live = 2592000; --  
    30 days TTL  
  
-- Querying the last hour for device ABC  
SELECT recorded_at, temperature,  
    humidity  
FROM iot_sensor_readings  
WHERE device_id = ? AND day = ?
```

Deep Dive: Partition Size & Bucketing

Problem: a sensor emitting 10 readings/second for 1 year creates ≈ 315 million cells in one partition — exceeding the recommended 100K cells/partition limit.

Solution — Time Bucketing: add a time component (day, week, or month) to the partition key. Each day's readings form a separate partition, keeping each partition at a manageable size ($\approx 864K$ cells/day for 10 Hz).

Queries spanning multiple days must issue one query per bucket and merge results in the application layer — a standard trade-off in Cassandra design.

TWCS Compaction

TimeWindowCompactionStrategy groups SSTables by time window and compacts only within the same window, ensuring further time bucketing with time bucketed TTL.

Cassandra Use Cases & Anti-Patterns

[OK] Ideal Use Cases

- **IoT sensor data** — time-bucketed, high-frequency writes from millions of devices with automatic TTL expiry
- **Chat / messaging systems** — partition by room ID + cluster by timestamp; sequential reads serve chat history instantly
- **User activity logs** — clickstreams, event logs, session tracking at massive scale
- **Leaderboards** — partition by game/category, cluster by score (DESC)
- **Inventory & catalog** — wide rows

[!] Anti-Patterns

- **Relationship-heavy queries** — “find all users who purchased X and also rated Y above 4 stars” requires scanning multiple partitions or application-side filtering
- **Ad-hoc analytics** — unknown WHERE clause patterns require ALLOW FILTERING, which scans all partitions — catastrophically slow at scale
- **Frequent updates to the same cell** — creates a chain of versions that slow down reads (compaction hasn't merged them yet)
- **Many small partitions** — millions of

Architectural Comparison: At a Glance

Dimension	Neo4j	Apache Cassandra
Category	Graph Database	Wide-Column Store
Origin	Sweden, 2007	Facebook, 2007 (Apache 2009)
CAP position	CA / CP (Raft-based)	AP (tunable)
Consistency model	ACID, full transactions	BASE, eventual (tunable to strong)
Topology	Master-follower (Core + Read Replicas)	Masterless ring (peer-to-peer)
Data model	Nodes, Relationships, Properties	Partitions, Rows, Clustering Columns
Query language	Cypher (openCypher / GQL)	CQL (SQL-like)
Write pattern	Raft-committed, ACID	Append to Commit Log + Memtable
Read pattern	Index-free adjacency pointer chain	Bloom filter + Partition index + SSTable merge
Scalability axis	Scale-up (+ Fabric for sharding)	Linear scale-out (add nodes to ring)
Fault tolerance	Raft quorum (majority must survive)	RF copies; any node can serve any request
Schema flexibility	High (schema-optional labels & props)	Moderate (per-table schema, strict partitioning)

Performance Characteristics: Write Path

Neo4j Write Performance

Strengths:

- Transactional integrity — complex multi-node/multi-relationship writes in a single ACID transaction
- Relationship creation is $O(1)$: append two relationship records and update two node pointers

Bottlenecks:

- Raft consensus adds latency: $\sim 5\text{--}50\text{ms}$ extra per write to achieve quorum
- Pessimistic locking can cause contention on “hot” nodes (supernode problem)
- Bulk loading of millions of nodes requires offline batch import tools (`neo4j-admin import`)

Cassandra Write Performance

Strengths:

- Writes are **sequential I/O** to Commit Log + Memtable — extremely fast ($<1\text{ms}$ latency at ONE)
- No locking: last-write-wins via timestamp means concurrent writes never block
- Linear throughput scaling: doubling nodes roughly doubles write capacity
- Benchmark: single node can sustain **100,000+ writes/second**

Bottlenecks:

- Tombstone accumulation from frequent deletes/updates degrades read performance
- Compaction I/O spikes can temporarily

Performance Characteristics: Read Path

Neo4j Read Performance

Strengths:

- **Deep traversal** — 6-hop friendship query on a 1B node graph completes in milliseconds via pointer chasing
- Traversal time is proportional to *result size*, not graph size
- In-memory page cache eliminates I/O for hot subgraphs
- Cypher planner optimizes complex pattern queries automatically

Bottlenecks:

- Analytical queries over the full graph (e.g., global PageRank) are memory-intensive; GDS library manages in-memory projections

Cassandra Read Performance

Strengths:

- **Single partition reads** are extremely fast: Bloom filter eliminates most disk seeks; clustering columns pre-sort data for sequential reads
- Time-series queries with LIMIT on a clustered partition are $O(1)$ — they read from the head of the sorted SSTable

Bottlenecks:

- **Multi-partition reads** — queries crossing partitions require coordinating multiple nodes; latency scales with partition count
- ALLOW FILTERING causes a *full table scan* — catastrophic at scale; always a design smell
- Read amplification increases with SSTable

Scalability: Scale-Up vs. Scale-Out

Neo4j: Vertical Scaling (Scale-Up)

- Core strength: adding RAM to fit more of the graph in the **page cache** provides near-linear read performance gains
- Raft Core Servers must be *identical in capability* — the weakest node determines consensus speed
- Horizontal write scaling via **Fabric** sharding, but requires application-level query routing and increased operational complexity
- Practical limit: a single Core Server can handle graphs of **~billions of nodes and relationships** if RAM is sufficient

Cassandra: Linear Scale-Out

- Adding a node to the ring **automatically redistributes** token ranges; capacity scales nearly linearly
- Both *reads and writes* benefit from adding nodes — unlike primary/replica architectures where only reads scale
- Netflix demonstrated linear scaling from 48 to 288 nodes with near-linear throughput increase
- Operational simplicity: `nodetool bootstrap / decommission` handle node lifecycle without downtime

Consistency & Durability Trade-offs

When strong consistency is non-negotiable:

- Financial transactions (double-spend prevention)
- Access control decisions (security policy changes must be immediately visible)
- Inventory management (two users must not both think the last item is available)

Neo4j handles these naturally via full ACID transactions. **Cassandra** can handle these via:

- QUORUM + QUORUM (strong consistency) — sacrifices some availability
- **Lightweight Transactions** (Paxos) — full linearizability, but 4× slower

Deep Dive: Last-Write-Wins Conflicts

Cassandra resolves concurrent writes using **timestamps**. The write with the higher timestamp wins. However:

- Clock skew across nodes (even with NTP) can cause the *earlier* write to win if its node's clock is ahead
- Solution: use **client-generated timestamps** and ensure monotonically increasing values
- The USING TIMESTAMP clause allows explicit control

For true conflict-free data structures, Cassandra 4.0+ introduced **CRDT-based counters**, but general-purpose CRDTs require application design.

Operational Complexity Comparison

Operation	Neo4j	Cassandra
Initial setup	Simple single-instance; Causal Clustering adds complexity	Requires cluster planning (RF, DC layout, vnodes) upfront
Schema changes	Add labels/properties without downtime; index creation is online	ALTER TABLE is instant (metadata only); data backfill is separate
Backup	Online backup via neo4j-admin backup; consistent snapshot	Snapshots per-node via nodetool snapshot; coordination needed
Monitoring	Built-in metrics endpoint, Prometheus integration, Neo4j Ops Manager	nodetool CLI, JMX metrics, DataStax Metrics Collector
Upgrades	Rolling upgrade supported (Core + Read Replica)	Rolling upgrade: drain, upgrade, restart node-by-node; fully online
Repair	Not required (Raft ensures consistency)	nodetool repair must run periodically to reconcile replicas
Tuning levers	Page cache size, heap, query plan hints	Compaction strategy, memtable size, CL per query, GC grace
Cloud-managed	AuraDB (fully managed, Neo4j Inc.)	Astra DB (DataStax), Amazon Keyspaces (AWS)

Developer Experience Comparison

Neo4j Developer Experience

- **Neo4j Browser / Bloom** — visual graph explorer; great for understanding data relationships intuitively
- **Cypher readability** — “I want users who bought X and rated Y” translates almost directly into a readable Cypher pattern
- **Schema flexibility** — add properties and labels to nodes without migration scripts
- **Driver ecosystem** — official drivers for Python, Java, JavaScript, Go, .NET; community drivers for many others

Cassandra Developer Experience

- **CQL familiarity** — SQL-like syntax lowers onboarding barrier for developers with SQL background
- **Strict modeling discipline** — query-first design requires upfront query enumeration; changing queries later may require new tables
- **Driver ecosystem** — DataStax Java Driver, Python Driver, Node.js Driver; all support async, connection pooling, speculative execution
- **Cassandra Workload Analyzer** — tool for identifying anti-patterns in CQL

Security Features Comparison

Neo4j Security

- **Authentication:** native username/password, LDAP, SSO (OIDC)
- **Authorization:** role-based access control (RBAC) with fine-grained privileges at database, graph, and property levels
- **Property-level security** (Enterprise): hide specific node/relationship properties from users without sufficient privilege
- **Subgraph-level security:** restrict which nodes/relationships a role can see
- **Encryption:** TLS for bolt/HTTP protocols; data-at-rest encryption via

Cassandra Security

- **Authentication:** internal (CassandraAuthenticator) or LDAP via DSE
- **Authorization:** object-level permissions (keyspace, table, function) via GRANT/REVOKE
- **SSL/TLS:** node-to-node and client-to-node encryption configurable independently
- **Transparent Data Encryption (TDE):** DSE Enterprise; encrypts SSTable files on disk
- **Audit logging:** introduced in Cassandra

Neo4j Ecosystem

- **GDS Library:** 65+ graph algorithms (centrality, community, pathfinding, ML embeddings)
- **APOC:** 450+ utility procedures
- **GraphQL API:** auto-generated from graph schema
- **Data connectors:** Kafka, Spark (GraphFrames), Elasticsearch
- **ETL tools:** Neo4j ETL tool for RDBMS migration; Data Importer (GUI)
- **Visualization:** Neo4j Bloom, yFiles, Graphistry (GPU-accelerated)
- **LLM integration:** LangChain Neo4j integration; Knowledge Graph + RAG patterns

Cassandra Ecosystem

- **Apache Spark (Cassandra Connector):** run Spark analytics directly over Cassandra data
- **Apache Kafka (Kafka Connect):** stream events into Cassandra at low latency
- **Presto/Trino:** SQL analytics layer over Cassandra for ad-hoc queries
- **Elasticsearch:** common pattern — Cassandra for primary store, ES for full-text search
- **Stargate:** API gateway providing REST, GraphQL, and Document APIs over Cassandra
- **Pulsar:** Apache Pulsar native Cassandra sink/source connector
- **Flink:** streaming analytics with Cassandra state store

Polyglot Persistence: The Modern Architecture Paradigm

Definition:

Polyglot persistence is the practice of using different data storage technologies within a single application to match each component's specific access patterns and data shape requirements.

Origin: Coined by Martin Fowler and Pramod Sadalage in *NoSQL Distilled* (2012).

Core principle: Abandon the idea of a single “one-size-fits-all” database. Instead:

- Identify the nature of each data domain
- Select the storage engine optimized for that domain's access pattern
- Accept operational overhead of managing multiple systems in exchange for each system performing at

Common Polyglot Stack

Relational	Core transactions (orders)
Redis	Sessions, caching, pub/sub
Elasticsearch	Full-text search
Neo4j	Recommendation, fraud
Cassandra	Events, logs, timeseries
S3 / Blob	Binary assets

Trade-off

Polyglot persistence increases **data consistency complexity** (cross-store transactions are not atomic) and **operational burden** (more systems to monitor and maintain). Microservices architectures absorb this naturally via

Combining Neo4j and Cassandra: Architecture Patterns

Pattern 1: Event Sourcing + Graph Projection

- All user events (clicks, purchases, ratings) are written to **Cassandra** as immutable event log (partition by user_id, cluster by event_time)
- An asynchronous consumer reads the Cassandra changelog and projects relevant relationship events into **Neo4j** (e.g., “User A purchased Product B” → creates [:PURCHASED] edge)
- Neo4j serves real-time graph traversal queries (recommendations, fraud detection) over the projected graph

Pattern 2: Write to Both, Query from Right

- Application writes to both stores in the same business transaction (eventually consistent across stores)
- Relationship queries → Neo4j; timeseries/bulk queries → Cassandra
- A Kafka topic acts as the bridge; consumers update each store independently

Deep Dive: Data Consistency Risk

Writing to two databases in the same business operation creates a **distributed transaction problem**. If the Cassandra write succeeds but the Neo4j write fails:

- The event log is correct (Cassandra)
- But the recommendation graph is missing an edge (Neo4j)

Mitigation strategies:

- 1 **Outbox pattern**: write to Cassandra first; publish event to Kafka; Neo4j consumer processes at-least-once
- 2 **Saga pattern**: implement compensating transactions for rollback
- 3 **Eventual consistency SLA**: accept a brief window where Neo4j lags Cassandra

Case Study: University Social Platform Architecture

“BK Connect” — a university social network:

Service 1: Recommendation Engine (Neo4j)

- Data: Student nodes, Course nodes, Faculty nodes
- Relationships: [:ENROLLED_IN], [:FRIEND], [:STUDIES_WITH], [:SHARES_FACULTY]
- Queries: “Who should I connect with?”, “What courses are popular in my network?”
- Latency requirement: <50ms for 3-hop traversals

Service 2: Messaging Service (Cassandra)

- Table: messages_by_room (partition: room_id, cluster: sent_at DESC)
- Write pattern: 10,000+ messages/second during exam periods
- Query: “Get last 50 messages in room X” — answered by a single partition sequential read

Service 3: Activity Feed (Cassandra)

- Denormalized per-user timeline table
- On each social action (post, like, comment), the event is fanned out to followers’ timelines in Cassandra
- Partition: (user_id, week_bucket); Cluster: activity_at DESC

Service 4: User Profiles (PostgreSQL)

- Core identity data with full ACID — Neo4j and Cassandra reference user_id as a foreign key concept

Integration Fabric

Apache Kafka bridges all services: profile up-

Decision Framework: Choosing the Right Database

Choose Neo4j when:

- Your primary questions are about *relationships* (“how are X and Y connected?”, “what is the shortest path?”)
- The graph structure is unknown at schema design time (exploratory graph analytics)
- You need ACID transactions over multi-entity graph operations
- Traversal depth matters more than raw throughput
- The domain is naturally graph-shaped: social networks, knowledge graphs, dependency trees, org hierarchies

Decision Checklist

	Neo4j	Cassandra
Relationship queries?	✓✓	×
High write throughput?	×	✓✓
ACID transactions needed?	✓✓	(✓)
Graph traversal queries?	✓✓	×
Time-series data?	×	✓✓
Flexible queries?	✓✓	×
Linear scale-out?	×	✓✓
Multi-DC active-active?	(✓)	✓✓
Schema flexibility?	✓✓	✓
Known access patterns?	✓	✓✓

Choose Cassandra when:

- Write throughput is the primary concern (>10K writes/second)
- Data access patterns are known and stable

Emerging Trends in Graph Databases

Graph + AI / LLMs

- **Knowledge Graph + RAG (Retrieval-Augmented Generation):** LLMs are augmented by Neo4j knowledge graphs that provide structured, relationship-aware context — reducing hallucinations
- **GraphRAG** (Microsoft Research, 2024): community detection on knowledge graphs provides hierarchical document summaries for LLM context
- **Graph Neural Networks:** Neo4j GDS provides GraphSAGE and FastRP embeddings that can be used as input features for ML classifiers

GQL: The ISO Graph Query Standard

- **GQL (ISO/IEC 39075)** was ratified in April 2024 — the first international standard for property graph query languages
- Heavily influenced by Cypher (openCypher), SQL/PGQ (SQL:2023), and PGQL
- Neo4j 5.x roadmap includes GQL compliance
- Signals a maturation of the graph database ecosystem comparable to what SQL did for relational databases in the 1980s

Emerging Trends in Wide-Column Stores

Cassandra 5.0: Storage-Attached Indexing (SAI)

- SAI (Storage-Attached Indexing) replaces the legacy SASI and secondary indexes with a new architecture that co-locates index data with SSTable files
- Enables efficient **non-partition-key filtering** — previously the Achilles heel of Cassandra data modeling
- Supports vector search (Ann indexes) — enabling Cassandra to serve as a **vector database** alongside its traditional workloads
- Opens Cassandra to similarity search for

Cassandra & the Serverless Future

- **DataStax Astra DB**: fully managed Cassandra with serverless billing (per-request, not per-node) — removes the operational burden of ring management
- **Amazon Keyspaces**: Cassandra-compatible managed service on AWS; separates compute and storage (S3-based)
- These serverless models decouple Cassandra's scalability benefits from its operational complexity — making it accessible to teams without dedicated DBA expertise

Research Frontiers: Graph Computing at Scale

Open research problems:

- **Distributed graph traversal:** efficiently traversing graphs partitioned across hundreds of nodes without expensive cross-partition hops (the “edge-cut” problem)
- **Dynamic graph algorithms:** maintaining PageRank, community detection results incrementally as edges are added/removed — without recomputing from scratch
- **Temporal graphs:** efficiently querying “what did the graph look like at time T?” — requires bitemporal graph models
- **Heterogeneous graph learning:** Graph Neural Networks that handle multiple node/edge types (as in property graphs) without collapsing to a homogeneous representation

Research Context: GNN on Property Graphs

Traditional GNNs (GraphSAGE, GCN) assume homogeneous graphs (one node type, one edge type). Real-world property graphs like Neo4j instances are **heterogeneous**: a node can be `:Student`, `:Course`, or `:Faculty`; an edge can be `[:ENROLLED_IN]` or `[:FRIEND]`.

Heterogeneous GNNs (HAN, HGT, R-GCN) handle multiple types but require graph-type-specific aggregation functions. Active research area with significant implications for fraud detection and biomedical knowledge graphs.

Best Practices: Neo4j

[OK] Do

- **Model the domain as-is:** let entities become nodes and business relationships become typed edges with properties
- **Use indexes at entry points:** always index the properties you use in the first MATCH clause
- **Leverage MERGE for idempotent loading:** prevents duplicate data during re-imports
- **Size the page cache first:** aim to hold the full node and relationship stores in RAM for best traversal performance
- **Use PROFILE:** always inspect the query

[!] Don't

- **Don't use Neo4j as a logging store:** it is not designed for sequential, flat data ingestion; every write creates relationship pointers — expensive at billions of events
- **Don't create supernodes carelessly:** a node with millions of relationships degrades traversal. Partition relationships by type, date, or status
- **Don't skip relationship types:** an untyped `-[r]->` relationship loses semantic value and forces runtime filtering

Best Practices: Cassandra

[OK] Do

- **Design tables for queries:** list all read queries first; design one table per query pattern
- **Choose partition keys that distribute evenly:** avoid partition keys that concentrate data on a few nodes (“hot spots”)
- **Use TTL for ephemeral data:** let Cassandra automatically expire sessions, caches, and rate-limit counters rather than writing explicit deletes (which become tombstones)
- **Run `nodetool repair` regularly:** keeps

[!] Don't

- **Don't use ALLOW FILTERING in production:** it triggers a full table scan that bypasses all partitioning logic — catastrophic at scale
- **Don't generate excessive tombstones:** frequent overwrites of nullable columns create read-amplifying tombstone chains; prefer TTL over explicit deletes
- **Don't ignore partition size:** aim for partitions between 1MB and 100MB. Very large partitions cause GC pressure and compaction stalls

Summary: Core Takeaways

Neo4j in One Paragraph

Neo4j is the tool you reach for when **relationships are the data**. Its property graph model and index-free adjacency make multi-hop traversals practically instant, independent of total graph size. Full ACID transactions ensure graph integrity. Cypher's ASCII-art syntax makes complex relationship queries readable. The cost: write throughput and horizontal scalability are more constrained than purpose-built distributed stores.

Cassandra in One Paragraph

Cassandra is the tool you reach for when **volume**,

The Design Philosophy Difference

Neo4j:

"Model the reality of your domain."

Cassandra:

"Model the queries your application needs."

Both are correct philosophies — for *different* classes of problems. Wisdom is knowing which class your problem belongs to.

Conclusion: The Bigger Picture

What this course should leave you with:

- There is **no universally best database**. Every system makes trade-offs that make it optimal for some workloads and terrible for others.
- The CAP theorem, ACID vs. BASE, and the relational vs. NoSQL divide are not just academic distinctions — they are **engineering decisions with real production consequences**.
- **Polyglot persistence** is the pragmatic synthesis: combine Neo4j's graph intelligence with Cassandra's distributed power, bridged by event streaming, to build systems greater than the sum of their parts.
- The emerging intersection of **graph databases + LLMs** (GraphRAG, Knowledge Graphs) is a significant research and industry frontier — highly relevant if you are pursuing graduate research in AI or distributed systems.

Further Reading

- *Graph Databases*, Robinson, Webber & Eifrem (O'Reilly)
- *NoSQL Distilled*, Fowler & Sadalage (Addison-Wesley)
- *Cassandra: The Definitive Guide*, Carpenter & Hewitt (O'Reilly)
- Brewer (2000): "Towards Robust Distributed Systems" (CAP theorem)
- Abadi (2012): "Consistency Tradeoffs in Modern Distributed Databases" (PACELC)
- Edge et al. (2024): "GraphRAG: Unlocking LLM Discovery on Narrative Private Data" (Microsoft)
- ISO/IEC 39075:2024 (GQL Standard)

Thank You

Questions & Discussion

Neo4j

*"If your data is a graph,
model it as a graph."*

Cassandra

*"Scale without
compromise on uptime."*

Lâm Khánh Trình · HCMUT · C03021 | Database Systems